

UTILITIES



GateOverflow Chatbot

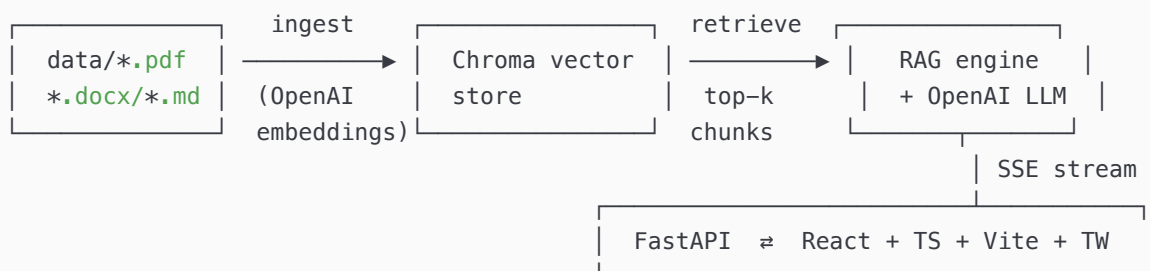
Source: README.md

Your AI study buddy for GATE CS & DA — grounded in real previous-year questions.

A production-ready **Retrieval-Augmented Generation (RAG)** assistant for **GateOverflow** (gateoverflow.in) that helps you prepare for:

- **GATE CS** — Computer Science & Information Technology
- **GATE DA** — Data Science & Artificial Intelligence
- ...and allied exams (ISRO, NIELIT, UGC-NET CS, TIFR)

Drop study material and previous-year-question (PYQ) PDFs into `data/`, the app indexes it, and the bot answers grounded in that material — with **streaming responses**, **source citations**, **conversation memory**, **LaTeX & code rendering**, and a beautiful **React + Tailwind** interface.



🌟 Features

- **RAG over your documents** — PDF, DOCX, PPTX, TXT, Markdown; recursive scan.
- **Rich knowledge base** — ships dense seed notes across the full CS & DA syllabus, plus a one-command fetcher for free/open textbooks, and ingests your own PYQ PDFs (CS, DA, ISRO, NIELIT, UGC-NET, TIFR).
- **Grounded answers with citations** — every answer can show source file, page/slide, similarity score and a snippet.
- **Broad, helpful scope** — answers across Computer Science, Data Science, ML and AI; politely declines only clearly-unrelated topics.

- **Conversation memory** — history-aware follow-ups (rewrites “what about its complexity?” into a standalone query).
- **Per-answer actions** — like 👍 / dislike 👎 / **copy** / **regenerate** on every turn, plus contextual **follow-up suggestion chips** — exactly like ChatGPT / Claude / DeepSeek.
- **Attachments** — 📎 attach **images** (a snapshot of a question is read by the vision model) or **PDF / text files** (extracted and used as context); paste images straight into the box.
- **Math & code rendering** — full **LaTeX** via KaTeX (auto-normalises $\backslash(…\backslash)$ / $\backslash[…\backslash]$ to $\$…\$$) and syntax-highlighted code blocks.
- **Natural small talk** — greetings like “hi” get a friendly welcome instead of a wrong retrieval-driven answer; clearly off-topic questions are politely declined.
- **GateOverflow look** — the familiar “GO + red ?” wordmark and brand colours.

🖋️ Study suite (beyond chat)

A full GATE preparation workspace, navigable from the sidebar:

- **Mock Test & Practice** — LLM-generated MCQ/MSQ/NAT, **server-scored with GATE negative marking**, estimated percentile, per-question explanations. Wrong answers auto-become flashcards.
- **Flashcards & Spaced Repetition** — **SM-2** scheduling; generate cards on any topic or review the ones created from your mistakes.
- **Study Planner** — a day-by-day plan to your exam date; **export to calendar (.ics)**.
- **Performance Dashboard** — readiness score, accuracy by subject, weak-area detection, percentile and streak.
- **Daily question + streaks** — a question of the day with a hint and a habit streak.
- **Socratic tutor mode, multilingual answers** (Hindi & more), **voice in/out** (Web Speech), **draw/sketch a question**, and a **confidence (grounding) meter** on every answer.
- **Local/offline model** support via `OPENAI_BASE_URL` (Ollama/LM Studio/vLLM).
- **Built-in guide** — an in-app “How to use” walkthrough (auto-opens on first visit; reopen via the ? button in the header) explains every feature and how to use it effectively.
- **GateOverflow PYQ links** — for previous-year-question answers (and *only* those), the exact GateOverflow question thread(s) are shown at the end of the answer, extracted from the URLs embedded in the PYQ PDFs (`python scripts/build_pyq_index.py` builds the lookup — no re-embedding).
- **Bookmarks & chat history** — ★ any answer to save it; revisit past conversations and saved answers from the 🕒 **history** panel in the header.
- **Cheat-sheet & PDF export** — build a revision cheat-sheet from a chat, or export the whole conversation, to **PDF** (one click → print/save).
- **Optional reranking** — set `RERANK=true` (with `flashrank`) for cross-encoder reranking of citations.
- **GateOverflow site awareness** — the assistant also knows the gateoverflow.in platform (pages, search syntax, PYQ database, PDFs/Book Viewer, test series, rank predictor, badges, how to ask) and can

guide users around it (system-prompt overview + a `data/seed/gateoverflow_platform_guide.md` knowledge file).

- **PYQ practice mode** — ask for previous-year questions by exam (GATE CS / DA, ISRO, NIELIT, UGC-NET, TIFR), get step-by-step solutions, and ask follow-ups.
 - **Feedback** → **RLHF loop** — every turn and rating is persisted; a disliked answer can be **regenerated steered by your feedback**, and corrections build a **preference dataset** you can export as JSONL for offline reward-model / DPO training. (See *Feedback & RLHF* below.)
 - **Streaming UX** — tokens appear as they're generated (Server-Sent Events).
 - **Modern frontend** — React 19 + TypeScript + Vite + Tailwind v4, dark/light mode, Markdown + KaTeX math + syntax-highlighted code, example prompts, expandable source cards. Fast, responsive, mobile-friendly.
 - **Persistence** — SQLAlchemy (SQLite by default, Postgres-ready) stores conversations, messages, feedback and preference pairs.
 - **Production API** — FastAPI with `/chat`, `/chat/stream` (SSE), `/feedback`, `/regenerate/stream`, `/export/preferences`, `/admin/stats`, `/health`, `/meta`.
 - **Incremental, resilient ingestion** — content-hash chunk IDs (no duplicates), a manifest that skips unchanged files (only re-embeds what changed), per-file error isolation, progress logging — built for large multi-hundred-page PDFs.
 - **Config via `.env`** — swap models, chunking, retrieval depth without code.
 - **Docker & docker-compose** — one command to run frontend + API.
-

■ Project structure

```
E.GATE-CS-DA-Chatbot/
├── api/main.py           # FastAPI backend (/chat, /chat/stream, /meta, /health)
├── scripts/
│   ├── ingest.py        # Build / refresh the vector index (CLI)
│   └── fetch_open_resources.py # Download free/open textbooks into data/
├── src/gate_chatbot/
│   ├── config.py        # Settings (env-driven)
│   ├── prompts.py       # Branding, system prompt, guardrail, templates
│   ├── ingestion.py      # Loaders, chunking, incremental indexing
│   ├── vectorstore.py    # Chroma + OpenAI embeddings
│   └── engine.py         # RAG orchestration (retrieve→guardrail→stream)
├── frontend/            # React + TS + Vite + Tailwind UI
│   ├── src/              # components, hooks, lib (SSE client)
│   ├── Dockerfile / nginx.conf # production build served by nginx
│   └── package.json
├── data/                 # ← your study material + PYQ PDFs (see data/README.md)
│   ├── seed/             # dense starter notes (shipped with the repo)
│   └── GATA-Data/         # your attached PYQ PDFs (CS, DA, ISRO, NIELIT, ...)
├── tests/test_smoke.py
├── requirements.txt
├── .env.example
├── Dockerfile / docker-compose.yml / Makefile
└── README.md
```

■ Quick start

1. Prerequisites

- **Python 3.10 – 3.12** (3.13+ may lack prebuilt `chromadb` wheels).
- **Node.js 18+** (for the frontend).
- An **OpenAI API key**.

2. Backend — install & configure

```
cd 18.Utilities/E.GATE-CS-DA-Chatbot

python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env          # then set OPENAI_API_KEY=sk-...
```

BASH

3. Add knowledge & build the index

Your PYQ PDFs already live in `data/GATA-Data/`. Optionally pull free/open textbooks too, then build the index:

```
python scripts/fetch_open_resources.py          # list open resources
python scripts/fetch_open_resources.py --download # (optional) download them

python scripts/ingest.py                        # incremental – only embeds new/changed files
python scripts/ingest.py --reset                # clean rebuild
python scripts/ingest.py --stats                # how many chunks are indexed?
```

BASH

First ingestion of the large PYQ PDFs (~137 MB) embeds many chunks and will take a few minutes and consume OpenAI credits. Subsequent runs skip unchanged files automatically (manifest-based incremental indexing).

4. Run the app

The app has **two parts** that must BOTH be running: the **API** (port 8000) and the **frontend** (port 5173). You open the **frontend** in the browser.

Option A — one command (recommended)

```
make dev          # or:  bash scripts/dev.sh
```

BASH

This starts the API **and** the frontend together (installing frontend deps on first run). Then open <http://localhost:5173>. Press `Ctrl+C` to stop both.

Option B — two terminals (manual)

```
# Terminal 1 – backend API (keep it running)
source .venv/bin/activate
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2 – frontend (keep it running)
cd frontend
npm install          # first time only
npm run dev
```

BASH

Now open <http://localhost:5173> — the dev server proxies `/api` → the API on port 8000. Start asking GATE questions. 🎉

⚠️ **Seeing nothing at <http://localhost:5173>?** That URL is the **frontend** — it only works while `npm run dev` (Terminal 2 / `make dev`) is running. Running only `uvicorn` starts just the API; the API itself lives at <http://localhost:8000/docs>, **not** 5173.

🔌 API reference

Method	Endpoint	Description
GET	<code>/health</code>	Liveness, model, indexed-chunk count
GET	<code>/meta</code>	Branding (name, tagline, scope) + status
POST	<code>/chat</code>	JSON answer + sources (synchronous, persisted)
POST	<code>/chat/stream</code>	SSE token stream; <code>done</code> event returns <code>conversation_id</code> + <code>message_id</code>
POST	<code>/feedback</code>	Record like/dislike (+ reason, comment, correction)
POST	<code>/regenerate/stream</code>	Regenerate an answer, steered by prior feedback
GET	<code>/export/preferences</code>	Download the RLHF preference dataset as JSONL
GET	<code>/admin/stats</code>	Conversation / message / feedback counters
POST	<code>/quiz/generate</code>	Generate a quiz/mock test (questions sent without answers)
POST	<code>/quiz/submit</code>	Score with GATE negative marking; returns explanations + analytics
POST	<code>/flashcards/generate</code>	Generate spaced-repetition cards on a topic
GET	<code>/review/due</code>	Cards due for review (SM-2)
POST	<code>/review/grade</code>	Grade a card (0–5) → reschedule with SM-2
POST	<code>/plan/generate</code>	Generate a day-by-day study plan
GET	<code>/plan</code> · <code>/plan/calendar.ics</code>	Latest plan (JSON) / calendar export
GET	<code>/daily</code>	Question of the day + streak
GET	<code>/analytics</code>	Readiness score, accuracy by subject, weak areas

BASH

```
curl -s http://localhost:8000/chat \  
  -H "Content-Type: application/json" \  
  -d '{"question":"Explain dynamic programming with an example.","history":[]}' | jq
```

Interactive docs at <http://localhost:8000/docs>.

■ Docker

BASH

```
docker compose up --build  
# Frontend → http://localhost:8080  (nginx serves the built React app)  
# API      → http://localhost:8000
```

nginx proxies the frontend's `/api/*` calls to the `api` service, so the browser talks to one origin. `data/` and `storage/` are mounted as volumes on the API so your documents and index persist. Ingest inside the container with:

BASH

```
docker compose run --rm api python scripts/ingest.py
```

Configuration reference (`.env`)

Variable	Default	Meaning
<code>OPENAI_API_KEY</code>	—	Required. Your OpenAI key.
<code>CHAT_MODEL</code>	<code>gpt-4o-mini</code>	Answer model (<code>gpt-4o</code> / <code>gpt-4.1</code> for best quality).
<code>EMBEDDING_MODEL</code>	<code>text-embedding-3-small</code>	Embedding model for retrieval.
<code>TEMPERATURE</code>	<code>0.2</code>	Lower = more deterministic.
<code>MAX_TOKENS</code>	<code>1024</code>	Max answer length.
<code>CHUNK_SIZE</code> / <code>CHUNK_OVERLAP</code>	<code>1000</code> / <code>150</code>	Chunking for indexing.
<code>RETRIEVER_K</code>	<code>5</code>	Chunks retrieved per question.
<code>RELEVANCE_THRESHOLD</code>	<code>0.25</code>	Min similarity to treat a question as grounded (controls the “answered from general knowledge” note).
<code>COLLECTION_NAME</code>	<code>gate_cs_da</code>	Chroma collection name.
<code>API_HOST</code> / <code>API_PORT</code>	<code>0.0.0.0</code> / <code>8000</code>	API bind address.

Frontend build-time var (`frontend/.env`): `VITE_API_BASE` (production API URL).

Changed the embedding model or chunk size? Re-index with `python scripts/ingest.py --reset`.

How it works

- Ingestion** (`ingestion.py`) — each file is parsed to text, split into overlapping chunks, tagged with provenance (`source` , `subject` , page/slide), embedded with OpenAI and stored in **Chroma** with content-hash IDs. A JSON manifest records each file’s size+mtime so unchanged files are skipped.
- Query reformulation** (`engine.py`) — a follow-up is rewritten into a standalone query using recent chat history.
- Retrieval** — top-`k` chunks by cosine similarity; the best score informs whether the answer is grounded in indexed material.
- Generation** — the system prompt keeps the assistant focused on CS / DS / ML / AI and GATE prep, injects retrieved context, and streams a grounded, exam-oriented answer.

5. Citations — de-duplicated sources are returned alongside the answer.

Why RAG and not fine-tuning?

RAG keeps answers **current and source-grounded**, lets you **add/update material instantly** (just re-ingest), avoids costly retraining, and reduces hallucination by anchoring responses to your documents. Fine-tuning would lock knowledge into weights and need retraining on every syllabus update.

Feedback & RLHF

Every question, answer and rating is stored (SQLAlchemy). The app implements the practical, industry-standard **RLHF** workflow used by production assistants — collect human preferences, then improve the model — split into an *online* and an *offline* half:

Online (immediate):

- 👍 on an answer opens a quick feedback form (reason + comment + an optional “better answer”). Hitting **Regenerate** then calls `/regenerate/stream`, which feeds your dislike reason/comment back into the prompt so the model produces an improved, feedback-conditioned answer.

Offline (training data → model update):

- A **correction** to a disliked answer, or a regeneration that replaces a disliked one, is stored as a **preference pair** (`prompt`, `chosen`, `rejected`) in the `preference_pairs` table.
- Export them anytime as JSONL:

```
curl -s http://localhost:8000/export/preferences -o preferences.jsonl
```

BASH

This is exactly the format consumed by a **reward model** or **DPO** fine-tune (`TRL DPOTrainer` / `RewardTrainer`, or OpenAI preference fine-tuning). Train offline, then point `CHAT_MODEL` at the improved model and redeploy.

Full reward-model + PPO training is an offline process — this app provides the complete human-feedback **data collection + online steering** loop and the export pipeline that feeds it, which is the realistic RLHF phenomenon for a deployed RAG product.

Data model (`src/gate_chatbot/db.py`): `conversations` → `messages` → `feedback`, plus `preference_pairs`.

■ Tests

BASH

```
pytest -q # offline backend smoke tests (no API key needed)
cd frontend && npm run build # type-checks + builds the frontend
```

■ Notes for production

- **Secrets:** keep `OPENAI_API_KEY` in `.env` / a secret manager — `.env` is git-ignored.
- **CORS:** `api/main.py` allows all origins for convenience — restrict to your front-end domain before deploying (or rely on the nginx same-origin proxy).
- **Auth & rate limiting:** add an API gateway / auth layer in front of FastAPI.
- **Persistent vector store:** for multi-instance deployments, point Chroma at a shared volume or migrate to a hosted vector DB.
- **Cost control:** `gpt-4o-mini` + `text-embedding-3-small` is the cheap default; monitor usage on the OpenAI dashboard.
- **Copyright:** only ingest material you're licensed to use (see `data/README.md`).

■ Roadmap ideas

- Per-subject / per-exam filtering in the UI
- Reranking (cross-encoder) for sharper retrieval
- Citation click-through to the source page/PDF
- Evaluation harness with PYQ benchmark questions
- User accounts, saved chats, and shareable answers

Built for **GateOverflow** · part of the **AI-ML-DS** workspace · `18.Utilities/E.GATE-CS-DA-Chatbot`